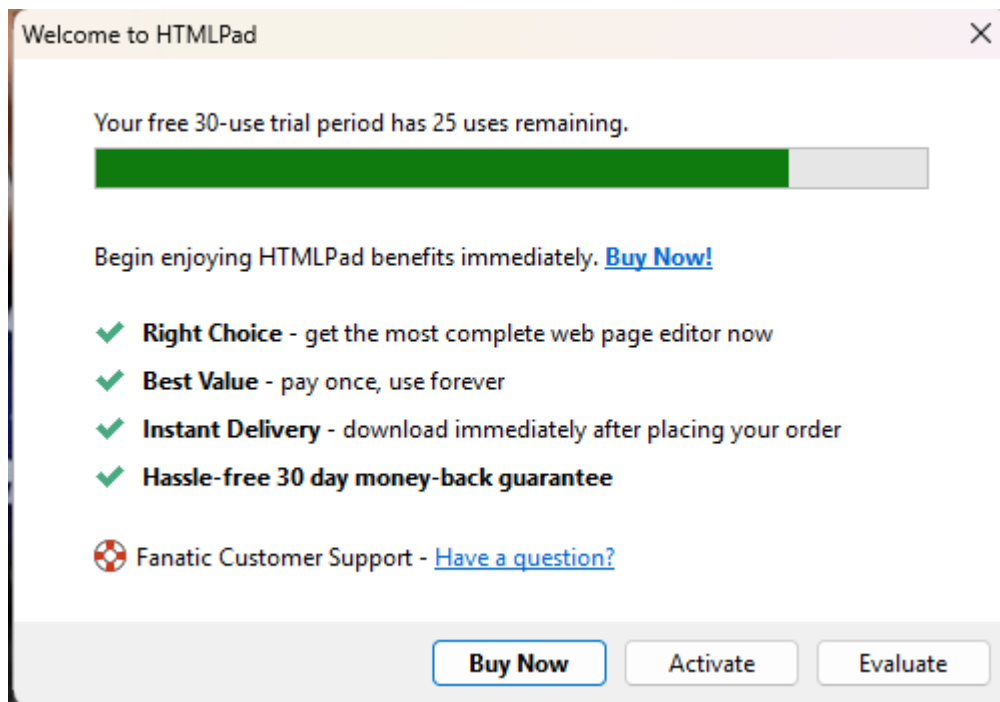


PHƯƠNG PHÁP THỰC HIỆN

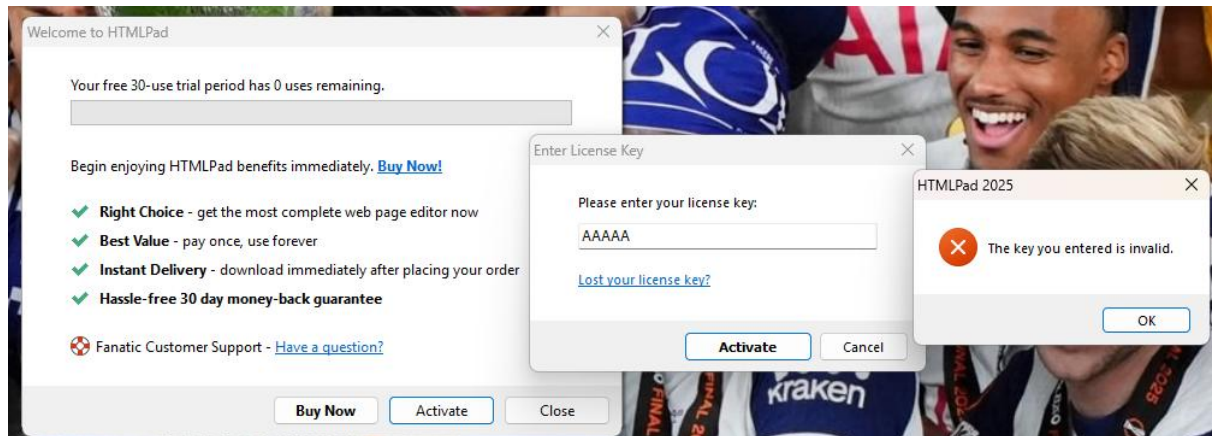
Giai đoạn 1: Khảo sát phần mềm

- Mục tiêu: Nắm bắt các phản ứng và cơ chế cảnh báo của hệ thống trước khi can thiệp.
- Thao tác: Tiến hành điều chỉnh thời gian hệ thống tăng thêm 1 tháng để ép phần mềm hiện thông báo hết hạn. Đồng thời, thử nghiệm nhập khóa bản quyền giả mạo và ghi nhận thông báo lỗi "The key you entered is invalid" để có cơ sở tìm kiếm luồng lệnh sau này.



Hình 1: Minh họa giao diện Trial mặc định

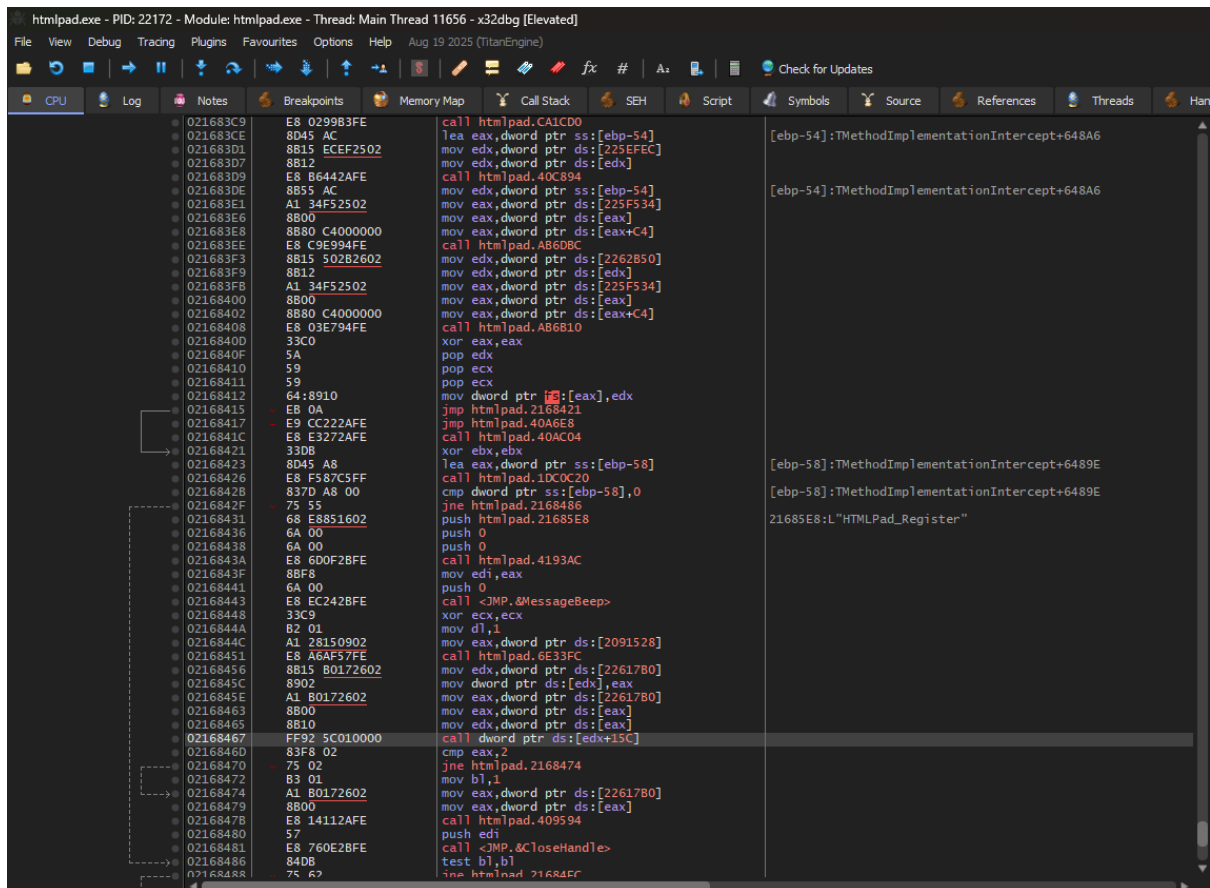
Khi hộp thoại thông báo thời gian dùng thử đã hết (0 uses remaining), và nếu nhập một khóa bản quyền giả mạo (ví dụ: "AAAAA"), phần mềm sẽ trả về thông báo "The key you entered is invalid".



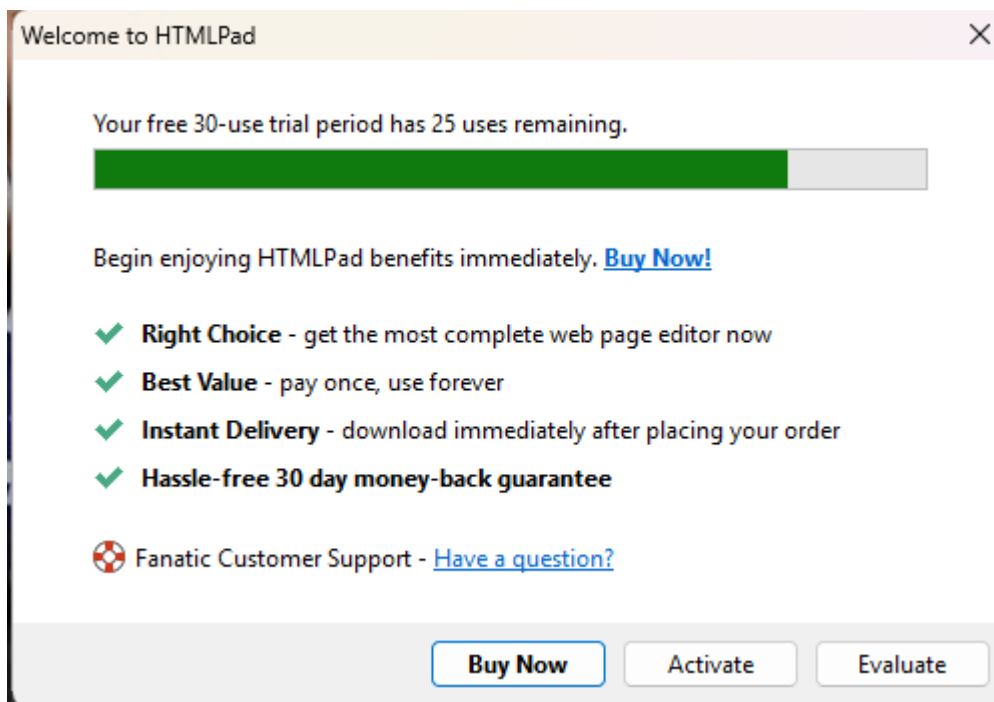
Hình 2: Thông báo hết hạn và nhập key sai

Giai đoạn 2: Phân tích và vô hiệu hóa giới hạn dùng thử (Bypass Trial)

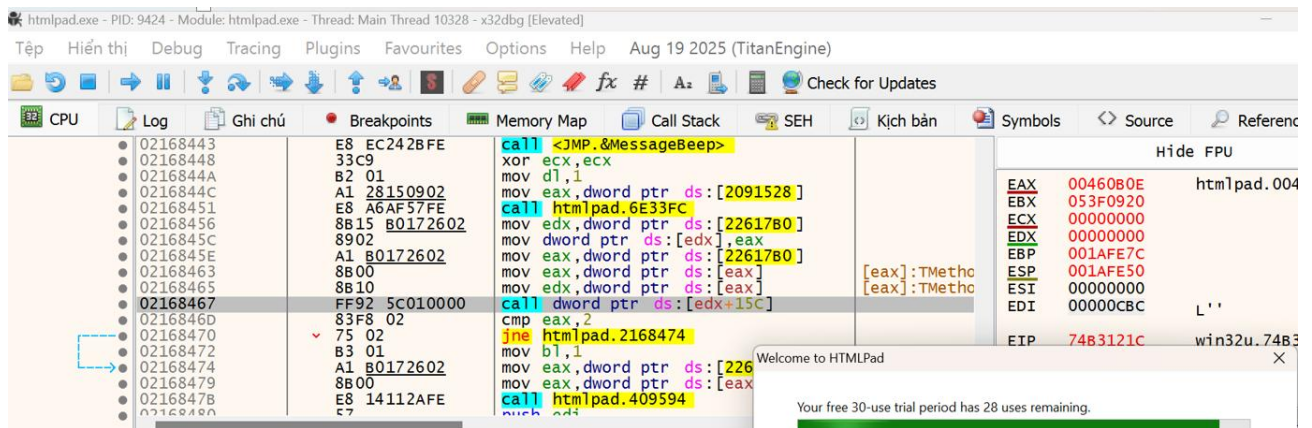
Mục tiêu: Xác định vị trí kiểm tra bản quyền và thay đổi luồng thực thi để vượt qua giới hạn 30 lần dùng thử Phân tích luồng thực thi trong x32dbg



Hình 3: Quá trình tracing và xác định lệnh CALL tại offset 02168467)



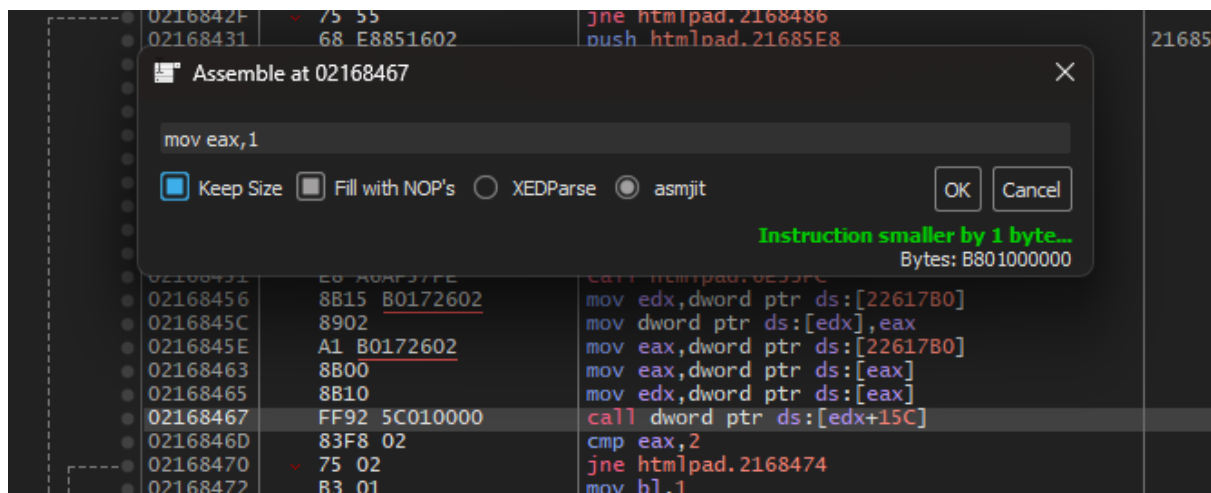
Hình 4: Quá trình tracing và xác định lệnh CALL tại offset 02168467)



Hình 5: Quá trình tracing và xác định lệnh CALL tại offset 02168467

Tiến hành đưa tệp tin htmlpad.exe vào trình gỡ rối x32dbg. Thông qua quá trình Tracing (Ctrl + F8), luồng thực thi dừng lại tại điểm gọi hộp thoại cảnh báo hết hạn dùng thử. Phân tích khu vực mã lệnh này, xác định được hộp thoại được kích hoạt từ một lệnh gọi hàm (CALL) tại địa chỉ offset 02168467, nhận giá trị trả về từ một hàm kiểm tra bản quyền trước đó là offset 02168451 .

Can thiệp mã lệnh (Patching)



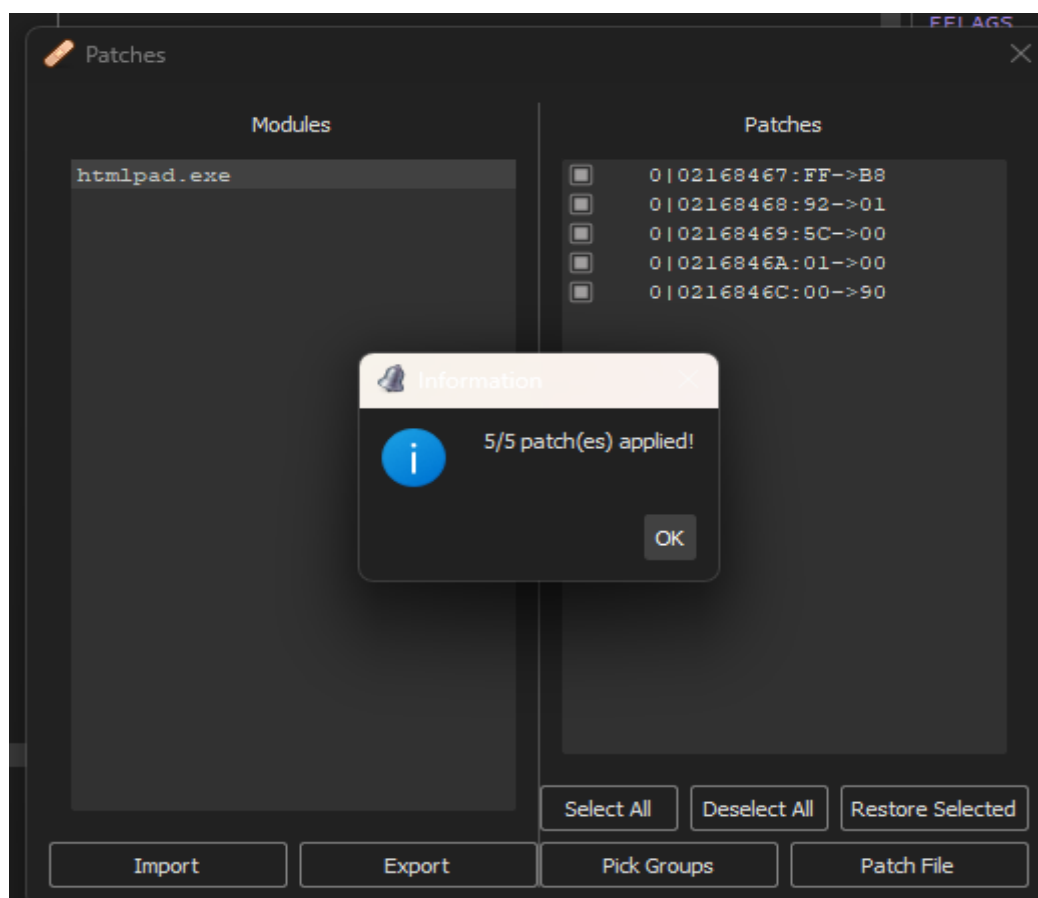
Hình 6: Thao tác sửa lệnh mov eax,1

0216844C	A1 28150902	mov eax,dword ptr ds:[2091528]
02168451	E8 A6AF57FE	call htmlpad.6E33FC
02168456	8B15 B0172602	mov edx,dword ptr ds:[22617B0]
0216845C	8902	mov dword ptr ds:[edx],eax
0216845E	A1 B0172602	mov eax,dword ptr ds:[22617B0]
02168463	8B00	mov eax,dword ptr ds:[eax]
02168465	8B10	mov edx,dword ptr ds:[eax]
02168467	B8 01000000	mov eax,1
0216846C	90	nop
0216846D	83F8 02	cmp eax,2
02168470	75 02	jne htmlpad.2168474

Hình 7: Thao tác điền NOP

Tại địa chỉ 02168467, thay vì để chương trình thực thi hàm kiểm tra gốc, ta tiến hành thay đổi mã lệnh trực tiếp (Assemble). Lệnh CALL được thay thế bằng lệnh gán giá trị trực tiếp `mov eax, 1` (mã hex: B8 01 00 00 00). Việc ép thanh ghi EAX mang giá trị 1 nhằm mục đích đánh lừa luồng kiểm tra logic bên dưới rằng ứng dụng đã ở trạng thái được đăng ký hợp lệ. Phần byte thừa còn lại của lệnh được thay thế bằng lệnh NOP (mã hex: 90 - No Operation) để đảm bảo cấu trúc lệnh không bị lệch.

Bước 4: Xuất bản vá

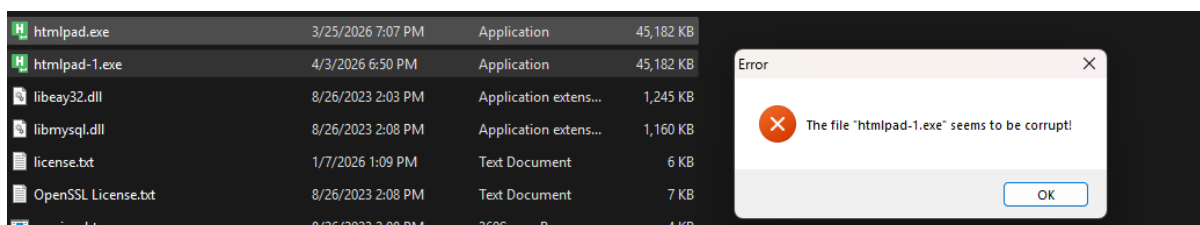


Hình 8: Xuất bản vá ra file mới

Lưu lại các thay đổi vào một tệp thực thi mới với tên `htmlpad-1.exe` (Patch file).

Giai đoạn 3: Phân tích và xử lý cơ chế tự bảo vệ (Defeating Anti-Tampering)

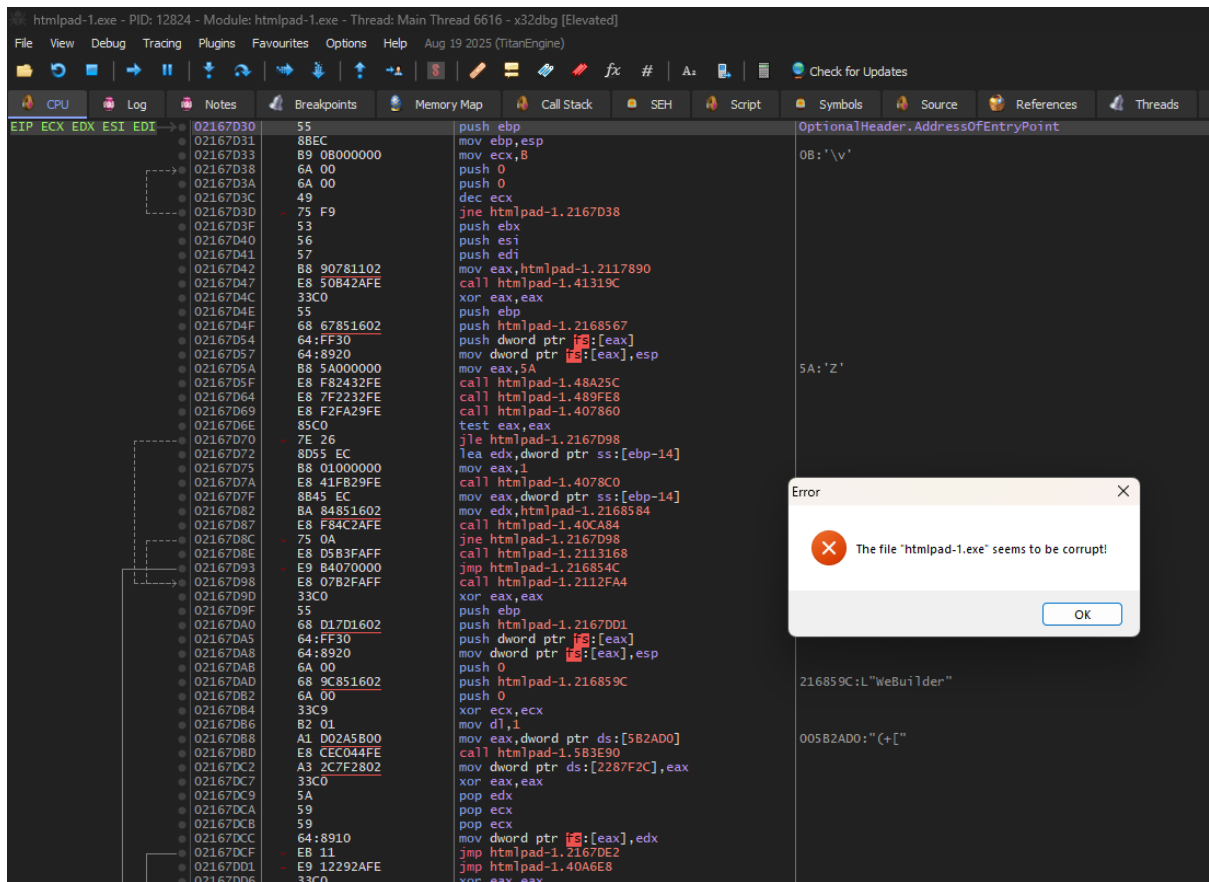
Mục tiêu: Khắc phục cơ chế kiểm tra tính toàn vẹn (Integrity Check) khiến phần mềm báo lỗi "corrupt file" và thoát đột ngột khi chạy file đã vá



Hình 9: Lỗi Corrupt khi mở file ngoài x32dbg

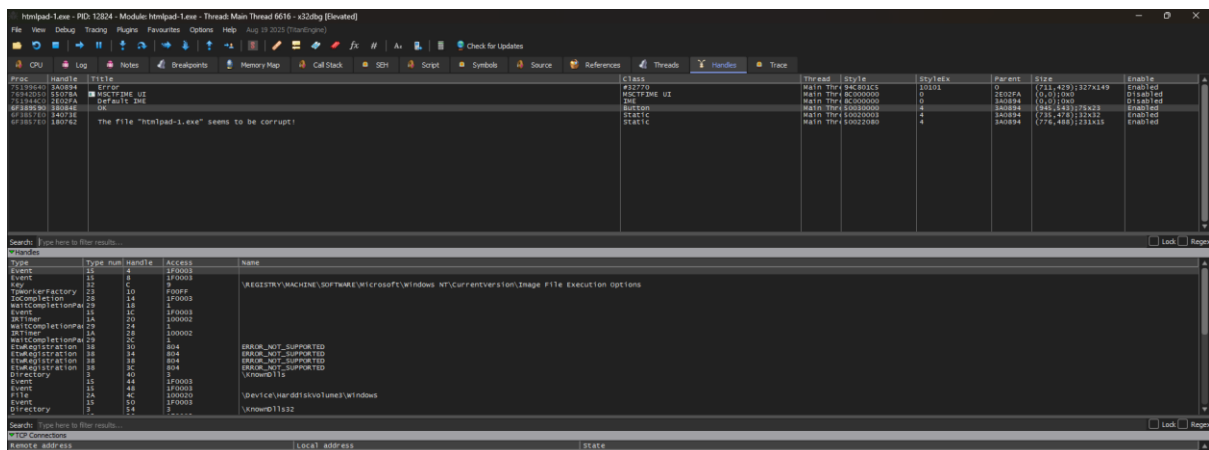
Khi khởi chạy tệp tin htmlpad-1.exe vừa được vá, phần mềm ngay lập tức phát hiện mã nhị phân của nó đã bị thay đổi và hiển thị hộp thoại lỗi: "The file 'htmlpad-1.exe' seems to be corrupt!". Đây là cơ chế kiểm tra tính toàn vẹn (Integrity Check) do chính phần mềm tự thực hiện nhằm chống lại việc dịch ngược. Để ngăn chặn, người lập trình cấu hình cho phần mềm hiện thông báo lỗi và buộc phải thoát ngay lập tức.

Nhà lập trình đã cấu hình để sau khi bấm OK, chương trình sẽ gọi ngay hàm hệ thống ExitProcess, trình gỡ rối không kịp dừng đúng lúc ở đoạn mã nguồn của HTMLPad mà lại nhảy thẳng vào các file hệ thống của Windows. Thay vì bắt khoảnh khắc bấm chuột, ta bắt khoảnh khắc phần mềm định thoát ra



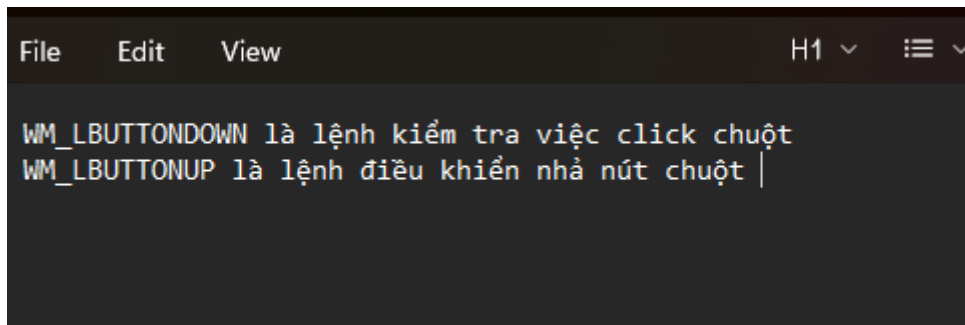
Hình 10: Chạy file trong trình gỡ rối lên bằng báo lỗi

Mở lại file htmlpad-1.exe trong x32dbg dưới quyền Administrator và cho chạy (F9) sẽ xuất hiện bảng thông báo lỗi.

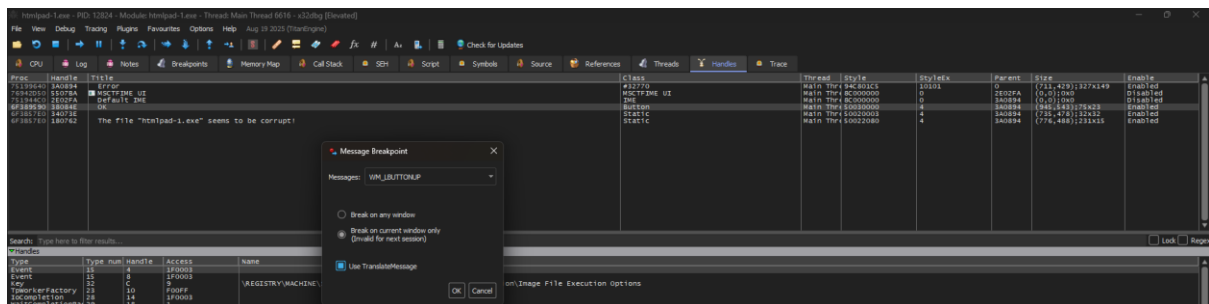


Hình 11: Cập nhật danh sách Handles của tiến trình trong x32dbg

Bảng điều khiển này được chương trình tạo ra trong quá trình thực thi, chứng tỏ rằng bảng thông báo lỗi là do cơ chế chống giả mạo kiểm soát tính toàn vẹn của ứng dụng cố ý tạo ra

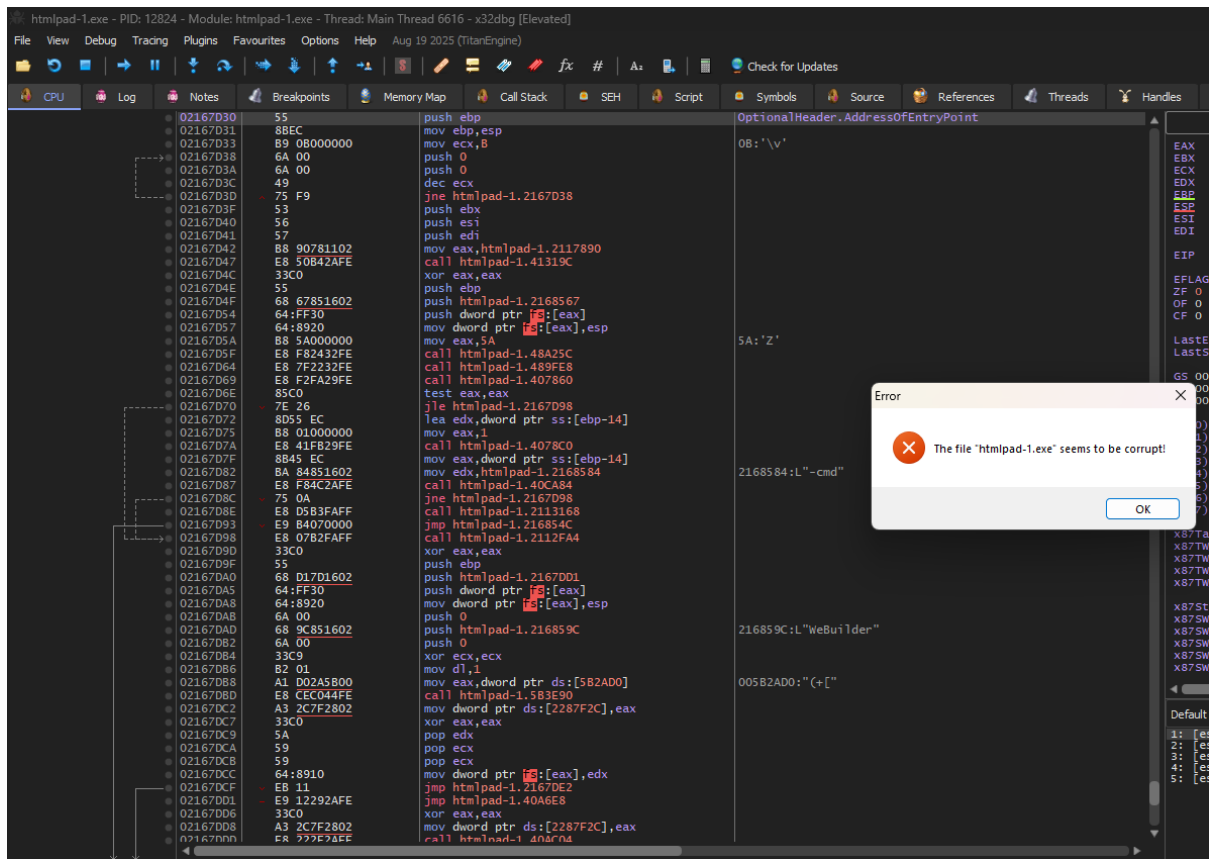


Hình 12: Chú thích về các thông điệp sự kiện tương tác chuột (Windows Messages)



Hình 13: Thiết lập điểm dừng thông điệp (Message Breakpoint) với sự kiện **WM_LBUTTONUP** cho nút OK

Khi bảng thông báo lỗi "corrupt" hiện lên, chương trình đang ở trạng thái chờ. Bằng cách đặt breakpoint vào sự kiện WM_LBUTTONUP (nhả chuột trái) trên nút OK, x32dbg dừng toàn bộ hệ thống ngay lập tức khi ngón tay vừa nhấc khỏi nút bấm đó, Nếu đặt ở WM_LBUTTONDOWN, sẽ dừng ngay khi vừa nhấn xuống. Khi dừng ở sự kiện click chuột, có thể dùng phím F8 (Step Over) để đi chậm từng dòng code bên trong htmlpad-1.exe, từ đó thấy được đoạn mã kiểm tra lỗi.



Hình 14: Chuyển về tab CPU và chuẩn bị kích hoạt sự kiện trên hộp thoại lỗi.

Sau khi nhấn OK và x32dbg dừng lại, bạn chuyển về Tab CPU. Lúc này, sẽ đứng ở vùng nhớ của hệ thống (DLL). Cần thực hiện Execute till user code (Alt+F9) hoặc nhấn F8 (Step Over) liên tục để thoát khỏi các hàm xử lý message của Windows và quay trở lại phân đoạn mã của htmlpad-1.exe.

Giai đoạn 4: Vá cơ chế báo lỗi toàn vẹn

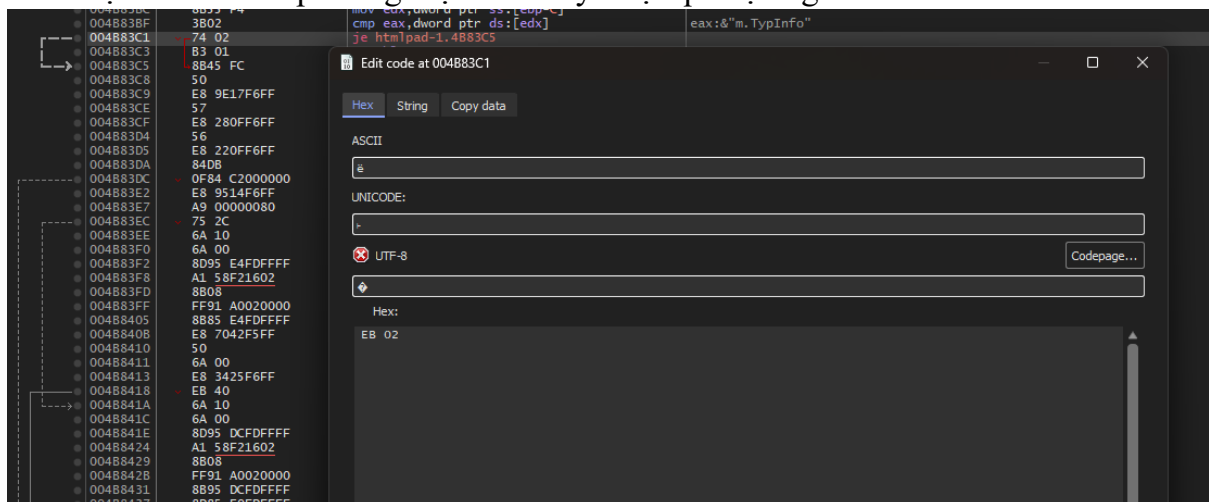
Phân tích logic bên trên điểm gọi MessageBox, ta nhận thấy chương trình sử dụng thanh ghi BL làm cờ (flag) để ghi nhận trạng thái lỗi. Nếu có lỗi, lệnh mov bl, 1 sẽ được gọi.

004883BC	8855 F4	mov edx,dword ptr ss:[ebp-C]	eax:&"m TypInfo"
004883BF	3802	cmp eax,dword ptr ds:[edx]	
004883C1	74 02	je htmlpad-1.4883C5	
004883C3	B3 01	mov bl,1	
004883C5	8B45 FC	mov eax,dword ptr ss:[ebp-4]	eax:&"m TypInfo"
004883C8	50	push eax	
004883C9	E8 9E17F6FF	call <JMP.&UnmapViewOfFile>	
004883CE	57	push edi	
004883CF	E8 280FF6FF	call <JMP.&CloseHandle>	
004883D4	56	push esi	
004883D5	E8 220FF6FF	call <JMP.&CloseHandle>	
004883DA	84D8	test bl,bl	
004883DC	0F84 C2000000	je htmlpad-1.488444	
004883E2	E8 9514F6FF	call <JMP.&GetVersion>	
004883E7	A9 00000080	test eax,80000000	eax:&"m TypInfo"
004883EC	75 2C	jne htmlpad-1.48841A	
004883EE	6A 10	push 10	
004883F0	6A 00	push 0	
004883F2	8D95 E4DFDFFF	lea edx,dword ptr ss:[ebp-21C]	[ebp-21C]:L"The file \"htmlpad-1.exe\" seems to be corrupt!"
004883F8	A1 58F21602	mov eax,dword ptr ds:[216F258]	eax:&"m TypInfo"
004883FD	8B08	mov ecx,dword ptr ds:[eax]	[eax]:"m TypInfo"
004883FF	FF91 A0020000	call dword ptr ds:[ecx+2A0]	
00488405	8B85 E4DFDFFF	mov eax,dword ptr ss:[ebp-21C]	[ebp-21C]:L"The file \"htmlpad-1.exe\" seems to be corrupt!"
0048840B	E8 7042F5FF	call htmlpad-1.40C680	
00488410	50	push eax	eax:&"m TypInfo"
00488411	6A 00	push 0	
00488413	E8 3425F6FF	call <JMP.&MessageBox>	
00488418	E8 40	jmp htmlpad-1.48845A	
0048841A	6A 10	push 10	
0048841C	6A 00	push 0	
0048841E	8D95 DCFDFFFF	lea edx,dword ptr ss:[ebp-224]	
00488424	A1 58F21602	mov eax,dword ptr ds:[216F258]	eax:&"m TypInfo"

Hình 15: Phát hiện lệnh nhảy và lệnh gán cờ lỗi mov bl, 1

Để vô hiệu hóa:

Tiến hành sửa đổi lệnh nhảy có điều kiện phía trên đoạn gán biến thành lệnh nhảy vô điều kiện EB 02 để ép luồng thực thi nhảy vượt qua lệnh gán lỗi.

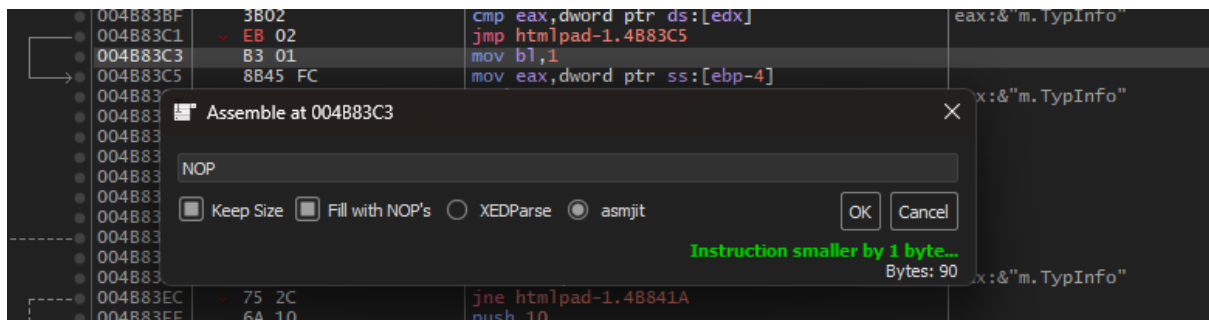


Hình 16: Sửa mã Hex thành EB 02

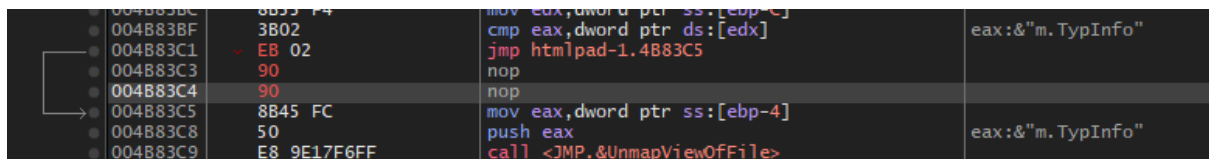
Tiếp tục thay thế triệt để lệnh mov bl, 1 bằng các lệnh NOP (mã hex: 90). Kỹ thuật này đảm bảo cờ BL luôn giữ giá trị 0 (không có lỗi), do đó khối lệnh gọi MessageBox cảnh báo và ExitProcess sẽ không bao giờ được kích hoạt.

004883BF	3802	cmp eax,dword ptr ds:[edx]	eax:&"m TypInfo"
004883C1	EB 02	jmp htmlpad-1.4883C5	
004883C3	B3 01	mov bl,1	
004883C5	8B45 FC	mov eax,dword ptr ss:[ebp-4]	eax:&"m TypInfo"
004883C8	50	push eax	
004883C9	E8 9E17F6FF	call <JMP.&UnmapViewOfFile>	
004883CE	57	push edi	
004883CF	E8 280FF6FF	call <JMP.&CloseHandle>	

Hình 17: Thao tác điền NOP thay cho lệnh gán lỗi

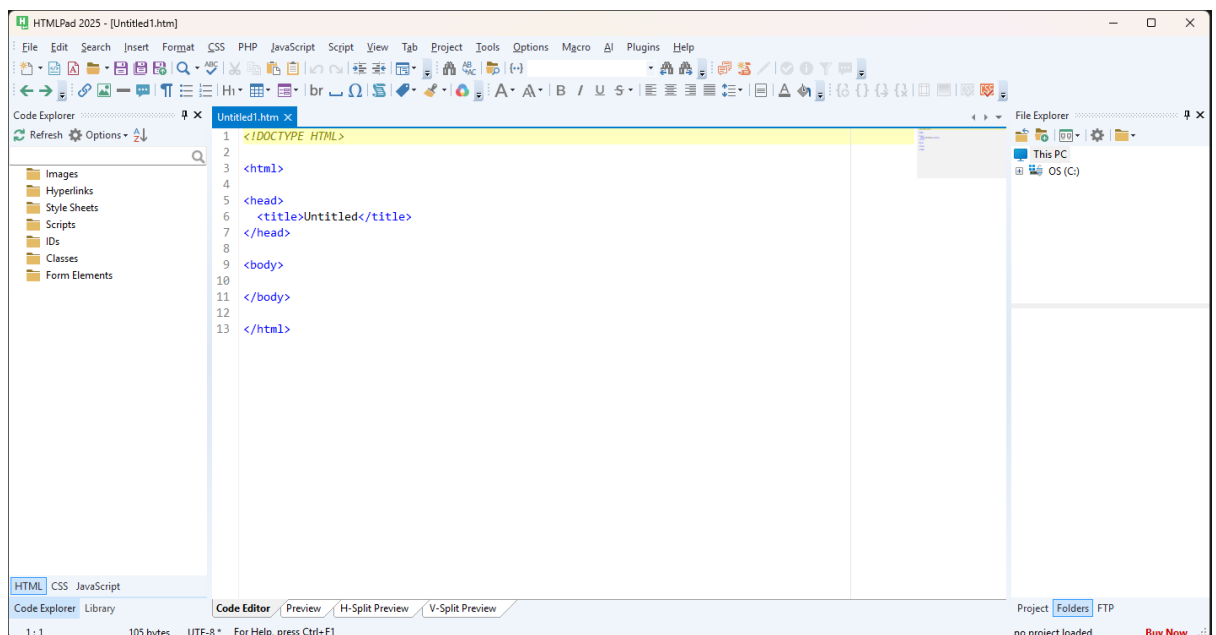


Hình 18: Thao tác điền NOP thay cho lệnh gán lỗi



Hình 19: Thao tác điền NOP thay cho lệnh gán lỗi

Sau khi lưu lại bản vá lần 2, phần mềm HTMLPad 2025 khởi chạy thành công, truy cập trực tiếp vào giao diện làm việc chính (Code Editor) mà không còn bất kỳ thông báo dùng thử hay lỗi toàn vẹn file nào. Quá trình dịch ngược và bẻ khóa hoàn thành.



Hình 20: Màn hình giao diện phần mềm đã bẻ khóa thành công